



OFICINA DE PROGRAMAÇÃO CALCULADORA



QUEM SÃO AS CODIFICADORAS?

Projeto de extensão da UTFPR-AP destinado a
incentivar mulheres na área de **computação**

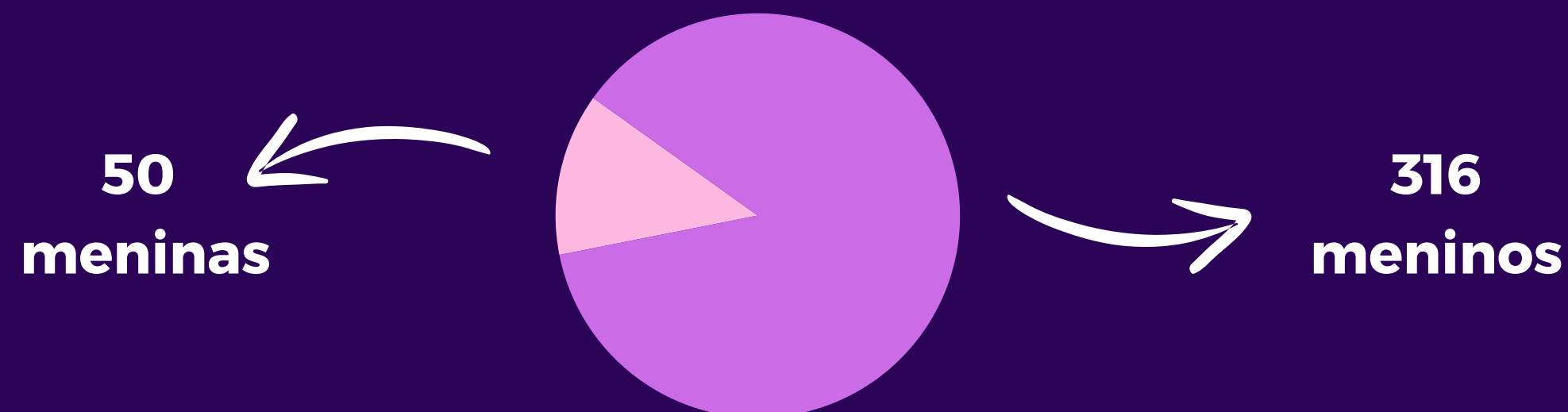




POR QUE ISSO É IMPORTANTE?

No curso de computação do nosso campus,
apenas 13%* dos alunos são mulheres

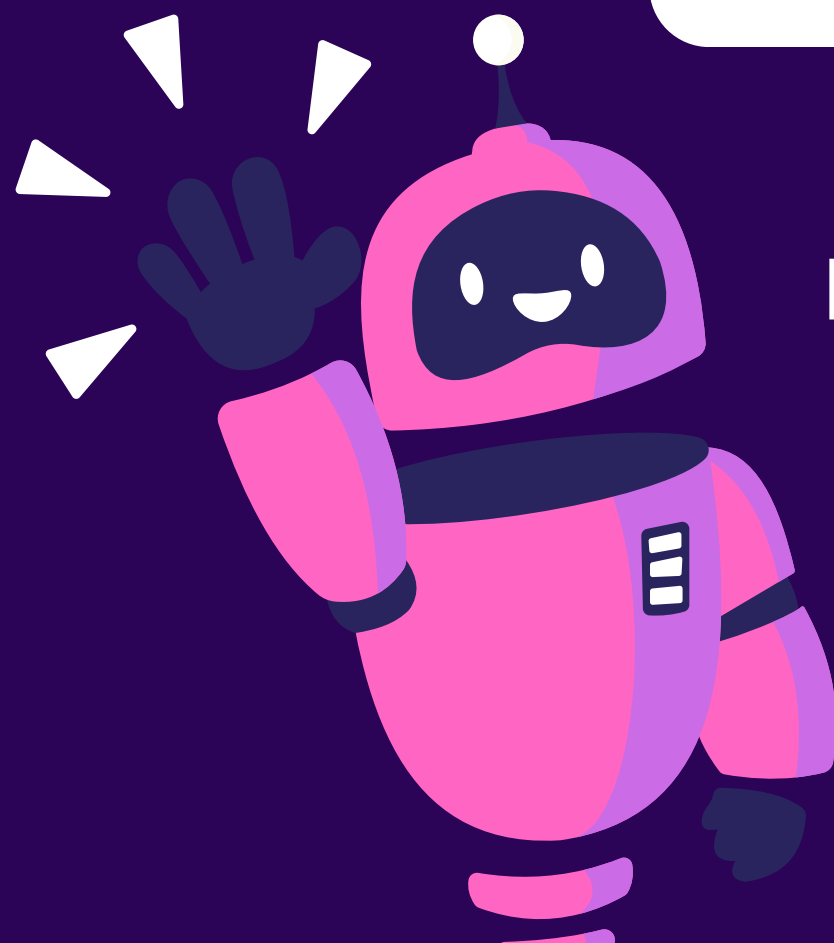
*Dados do Sistema da UTFPR de 2025





COMO PODEMOS MUDAR ISSO?

Computação também é **coisa de menina!**

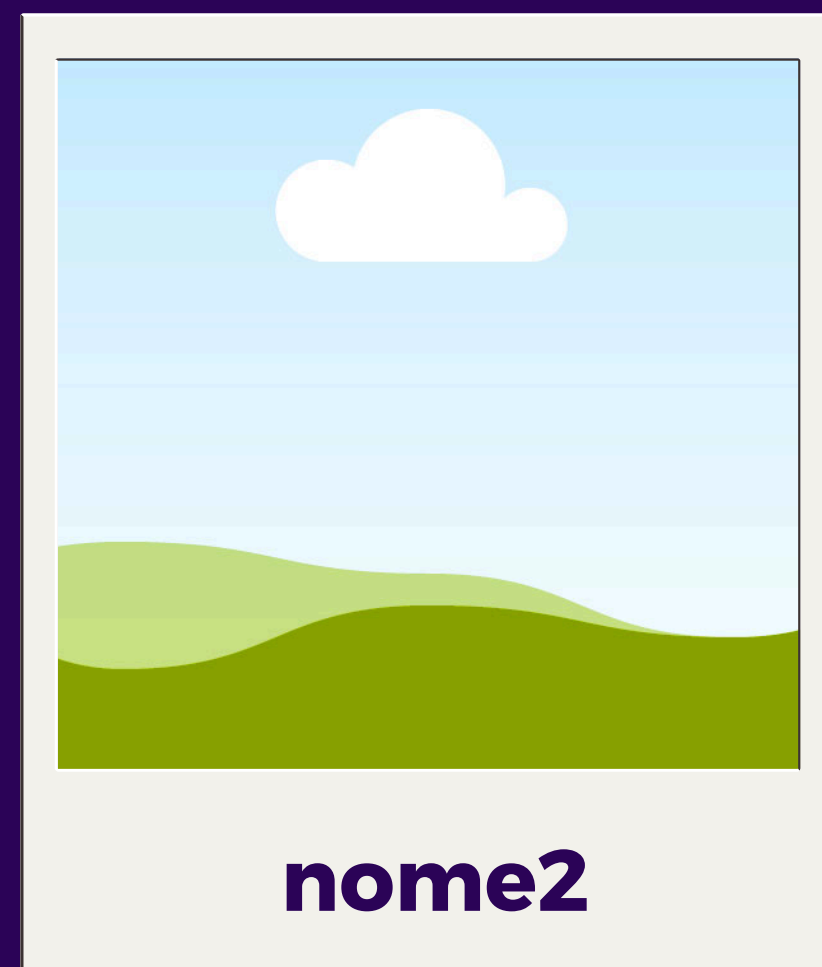
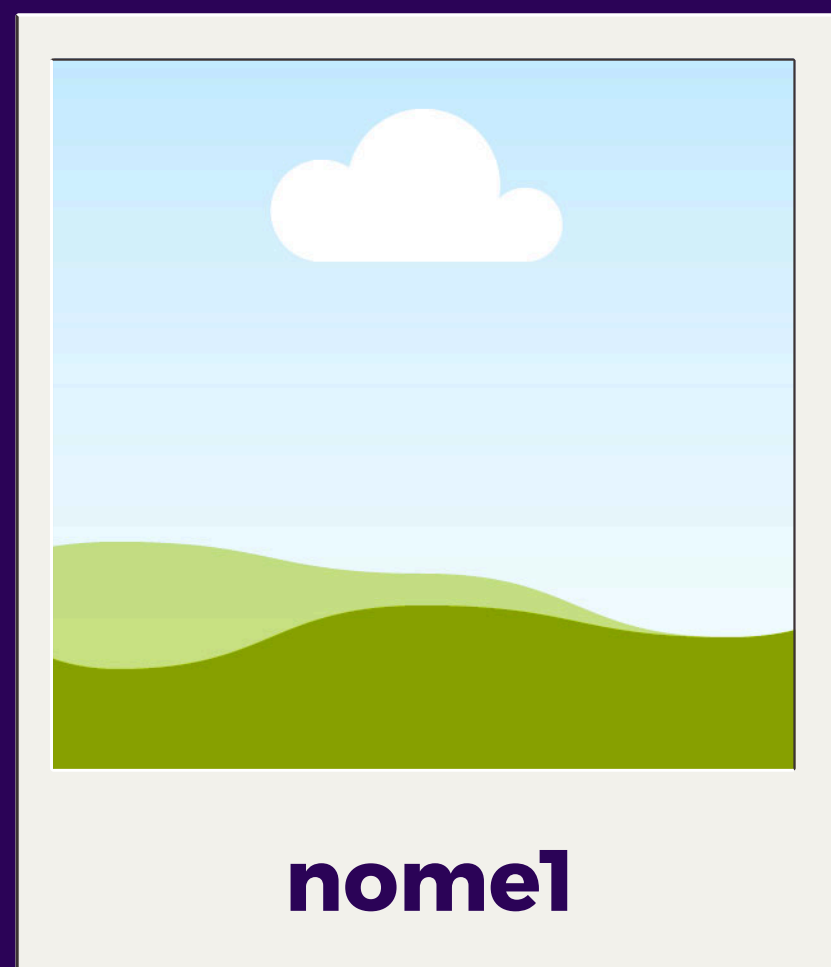


E nós iremos provar isso por meio de:

- Oficinas de Robótica
- Oficinas de Programação
- Divulgação do universo da Computação



MONITORAS DA OFICINA





**VOCÊ JÁ SE
PERGUNTOU COMO A
PROGRAMAÇÃO
PODE SER USADA NO
DIA A DIA?**





NO DIA A DIA

1

Automatizar tarefas

2

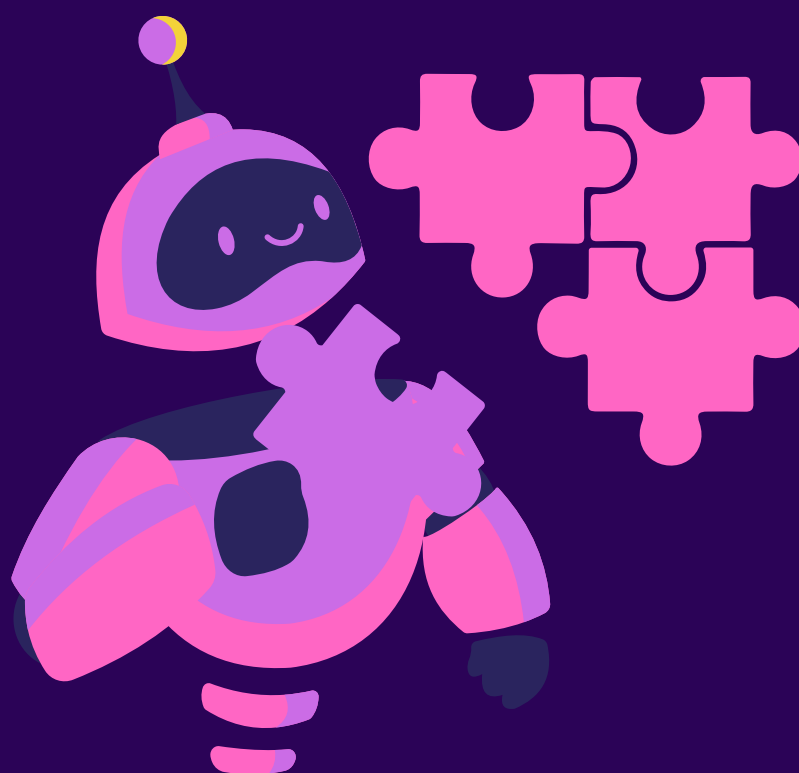
Jogos e diversão

3

Saúde e bem-
estar

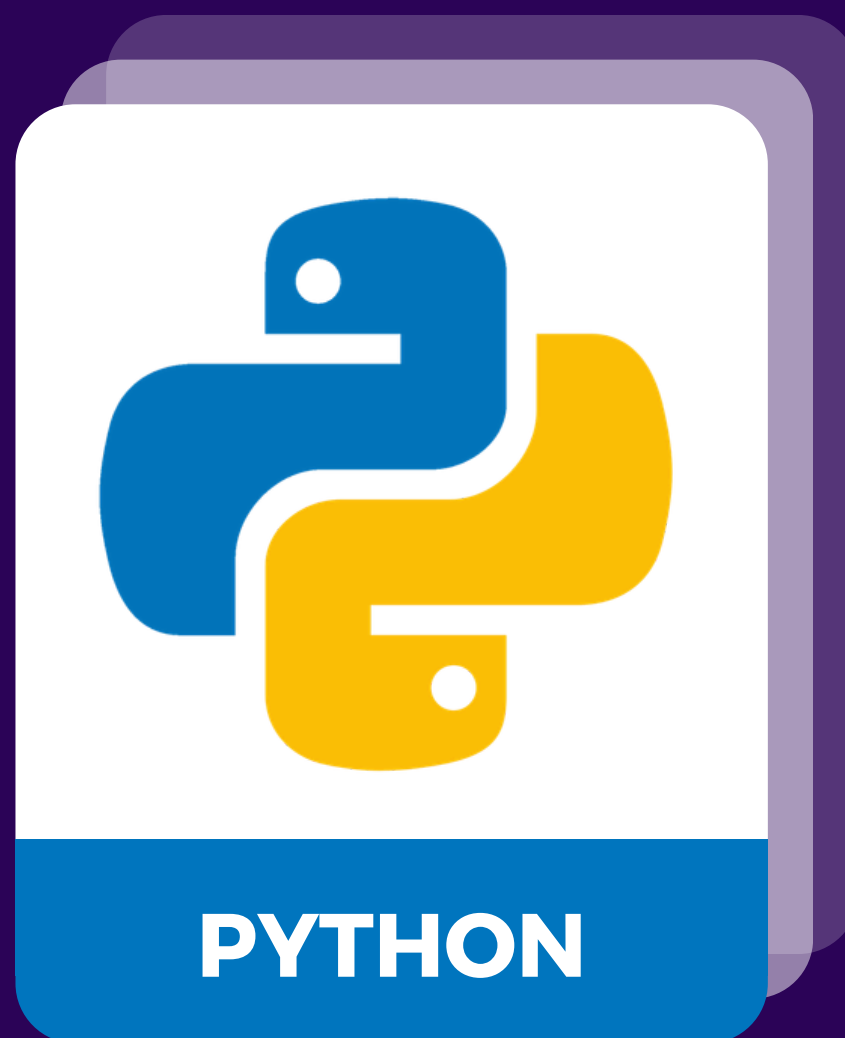


QUAIS SÃO AS FERRAMENTAS NECESSÁRIAS PARA CONSTRUIR UMA CALCULADORA?





FERRAMENTAS



Uma linguagem de programação é um conjunto de instruções que usamos para "conversar" com o computador.

Python é uma das linguagens de programação mais usadas no mundo. Ela é conhecida por ser simples, fácil de aprender e muito poderosa.



IMPRIMINDO NA TELA

Exemplo:

```
print("Sejam bem vindas")
```

Python



VARIÁVEL

Podemos **guardar nossos dados** em variáveis!

DADOS

DADOS

DADOS



VARIÁVEL

Tipos de dados que podemos
guardar em uma variável:



NÚMEROS



TEXTOS



**VALORES
BOOLEANOS**



DECLARANDO UMA VARIÁVEL

Exemplo:

Textos sempre devem ser
sinalizados por aspas

```
numero = 20  
nome = "Estefane"  
booleano = True
```

Python



INTRODUÇÃO À F-STRINGS

Exemplo:

```
nome = "Raissa"           # Python  
print(f"Olá, {nome}!")
```

Saída:

```
Olá, Raissa!              # Python
```




RECEBENDO UM VALOR DO USUÁRIO

Exemplo:

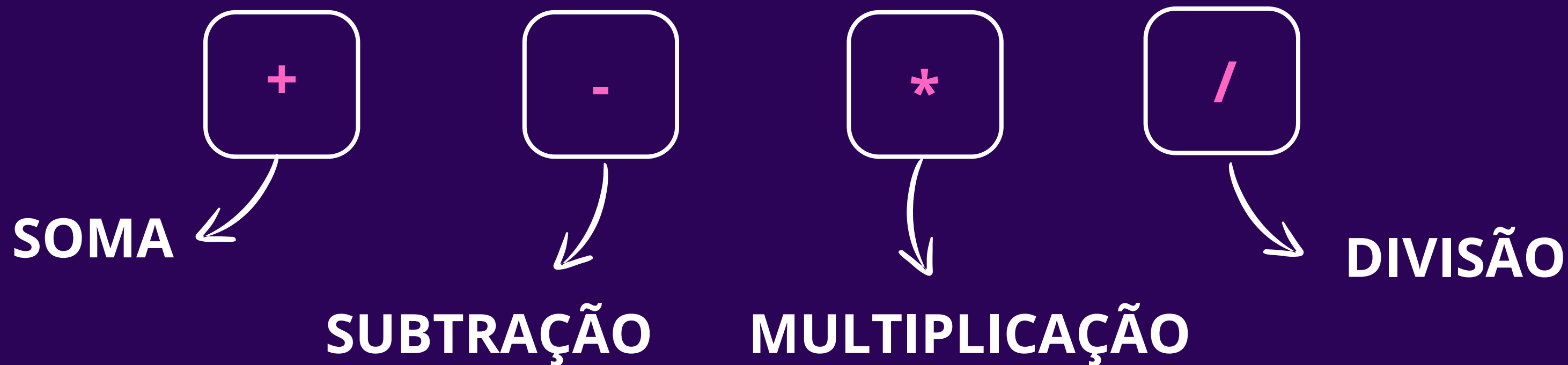
```
nome = input("Digite seu nome: ")          # Python
idade = int(input("Digite sua idade: "))

print(f"Olá {nome}, você tem {idade} anos!")
```



OPERAÇÕES ARITMÉTICAS

Podemos usar operações aritméticas para fazer **contas matemáticas** em nossos códigos!





ESTRUTURAS CONDICIONAIS

Toda vez que queremos que alguma coisa aconteça **se**
uma condição for atendida, utilizamos uma **estrutura**
condicional





OPERADORES LÓGICOS

Principais **operadores lógicos** em Python:

and

E

or

OU

not

NÃO

!=

DIFERENTE



OPERADORES COMPARATIVOS

Principais **operadores comparativos** em
Python:

==

IGUAL

>

MAIOR QUE

<

MENOR QUE



ESTRUTURAS CONDICIONAIS: IF/ELSE

Exemplo:

Se minha nota em matemática for maior ou igual a 6, serei aprovada. Se minha nota for menor que 6 e maior que 4 poderei fazer a recuperação, caso contrário, serei reprovada





ESTRUTURAS CONDICIONAIS: IF/ELIF/ELSE

Exemplo:

```
if nota >= 6:                                     # Python
    situação = "Aprovada"
elif nota >= 4 and nota < 6:
    situação = "Recuperação"
else:
    situação = "Reprovada"
```



TRY EXCEPT

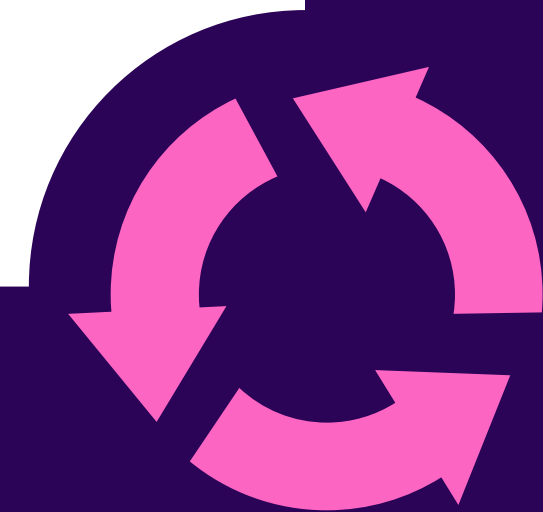
Exemplo:

```
try:                                     # Python
    numero = int(input("Digite um número: "))
except ValueError:
    print("A entrada recebida não é um número!")
    exit(1)
```




LAÇOS DE REPETIÇÃO

Toda vez que queremos que alguma coisa aconteça **várias vezes** em seguida, utilizamos um **laço de repetição**.





LAÇOS DE REPETIÇÃO

Componentes principais de um laço de repetição:

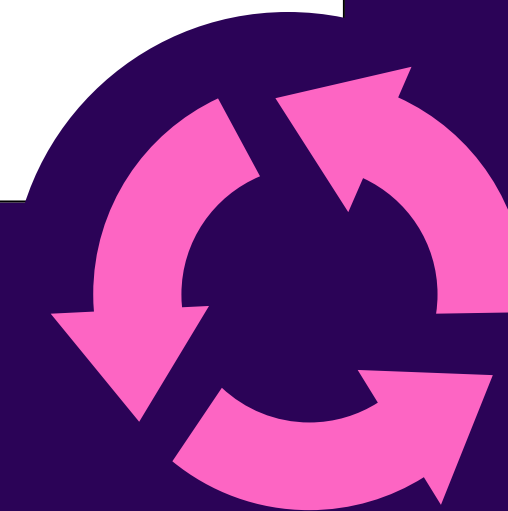
- variável de controle
- condição para interromper o laço de repetição
- atualização da variável de controle



LAÇOS DE REPETIÇÃO

Exemplo:

Mariana quer um buquê com **10** girassóis. Então, ela colhe **uma flor de cada vez** no jardim.





WHILE

Exemplo:

```
quantidadeGirassois = 0 # Python

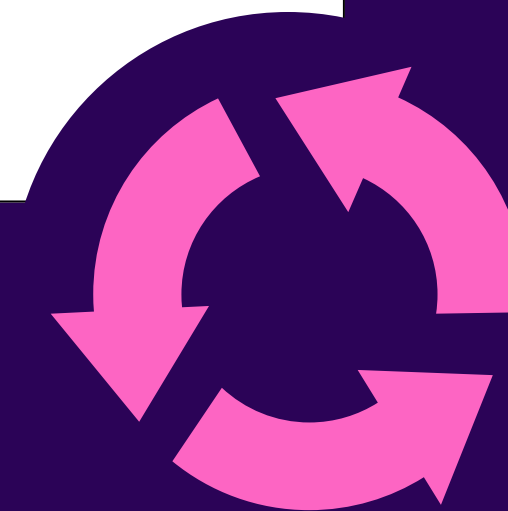
while quantidadeGirassois < 10:
    #Mariana realiza a colheita
    quantidadeGirassois += 1
```



LAÇOS DE REPETIÇÃO: INTERRUPÇÃO FORÇADA

Exemplo:

Allane está numa loja de roupas procurando **uma** saia, **mas** se ela achar **um** vestido, **não precisará** mais procurar a saia





WHILE: INTERRUPTÃO FORÇADA

Exemplo:

```
EncontrouSaia = False                                     # Python
EncontrouVestido = False

while EncontrouSaia == False

    EncontrouVestido = True
    if EncontrouVestido == True
        break
```



FUNÇÕES

Exemplo:

Bianca quer criar um programa para multiplicar $36 * 7$, $23 * 4$ e $54 * 8$. Como ela sabe que vai realizar a **mesma operação** em todos os casos, Bianca decidiu criar uma **função de multiplicação**.



FUNÇÕES

Podemos criar **funções** para **reaproveitar um bloco de código**



Isso evita repetições de código!



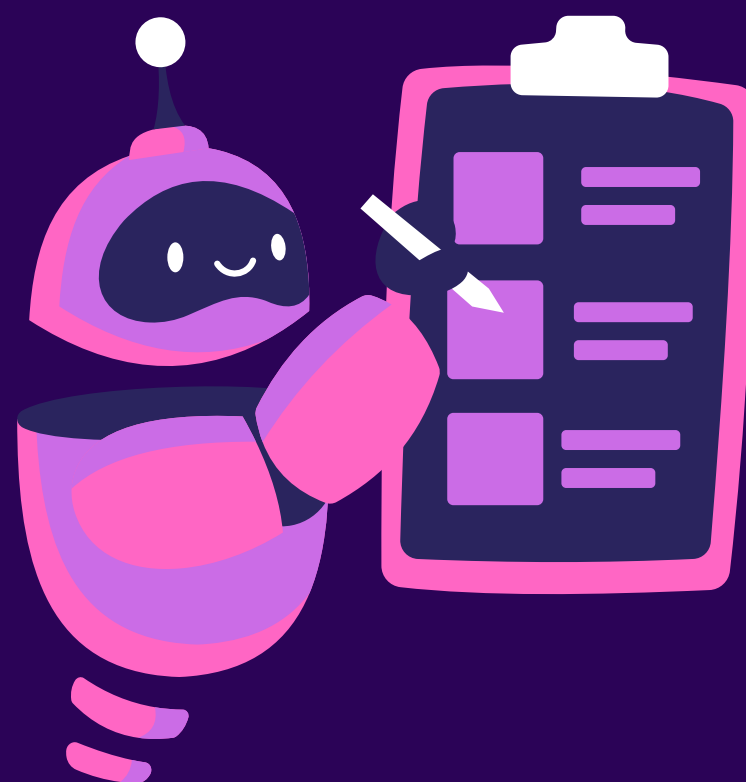
FUNÇÕES

Exemplo:

```
def multiplicacao(numero1, numero2):           # Python
    conta = numero1 * numero2
    return conta
```



UHUUUL, JÁ **SABEMOS** TUDO
O QUE É NECESSÁRIO PARA
FAZER UMA CALCULADORA!





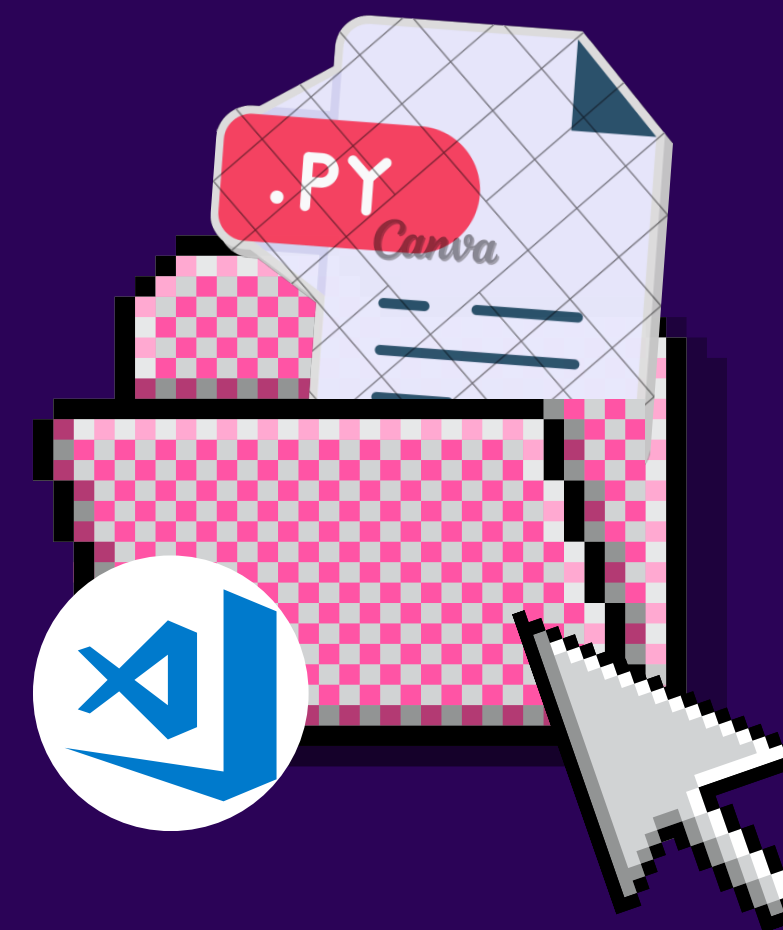
VAMOS COMEÇAR, FUTURAS CODIFICADORAS?





PREPARANDO NOSSO AMBIENTE

1. Acesse o aplicativo **Visual Studio Code**
2. Crie uma pasta para o **nosso projeto**
3. Crie um arquivo **main.py**
4. Crie um arquivo **funcoes_calculadora.py**





DECLARANDO AS FUNÇÕES

No arquivo que criamos `funcoes_calculadora.py`

```
def soma(numero1, numero2):  
    conta = numero1 + numero2  
    return conta
```

```
def subtracao(numero1, numero2):  
    conta = numero1 - numero2  
    return conta
```

```
def multiplicacao(numero1, numero2):  
    conta = numero1 * numero2  
    return conta
```

```
def divisao(numero1, numero2):  
    conta = numero1 / numero2  
    return
```



DECLARANDO AS FUNÇÕES

No arquivo que criamos `funcoes_calculadora.py`

```
def exponenciacao(numero1, numero2):  
    resultado = numero1 ** numero2  
    return resultado
```



ESTRUTURAÇÃO DA MAIN

Como podemos estruturar a **main** do nosso projeto?





ESTRUTURAÇÃO DA MAIN

Passo 1: Loop while para usarmos a calculadora até apertarmos o botão de sair

Passo 2: Exibimos um menu de operações

Passo 3: Receber a opção que o usuário digitou

Passo 4: Recebemos os números

Passo 5: Chamamos a função e exibimos o resultado





PASSO 1: LOOP WHILE

```
while True:  
    ... #Aqui vai ficar o conteudo
```



PASSO 2: MENU DE OPÇÕES

```
#Estamos dentro do While  
print("1 - Soma")  
print("2 - Subtração")  
print("3 - Multiplicação")  
print("4 - Divisão")  
print("5 - Sair")
```



PASSO 3: RECEBENDO A OPÇÃO

```
#Estamos dentro do While  
opcao = int(input("Escolha uma opção: "))  
  
if opcao == 5:  
    break
```




PASSO 4: RECEBENDO OS NÚMEROS

```
#Estamos dentro do While  
numero1 = float(input("Digite o primeiro número: "))  
numero2 = float(input("Digite o segundo número: "))
```



PASSO 5: CHAMANDO AS FUNÇÕES E EXIBINDO O RESULTADO

```
#Estamos dentro do While
if opcao == 1:
    resultado = funcoes.soma(numero1, numero2)
    print(f"{numero1} + {numero2} = {resultado}")
elif opcao == 2:
    resultado = funcoes.subtracao(numero1, numero2)
    print(f"{numero1} - {numero2} = {resultado}")
```



PASSO 5: CHAMANDO AS FUNÇÕES E EXIBINDO O RESULTADO

```
#Estamos dentro do While
elif opcao == 3:
    resultado = funcoes.multiplicacao(numero1, numero2)
    print(f"{numero1} * {numero2} = {resultado}")
elif opcao == 4:
    resultado = funcoes.divisao(numero1, numero2)
    print(f"{numero1} / {numero2} = {resultado}")
```



PRONTOS PARA A SEGUNDA ETAPA?





SEGUNDA ETAPA: NOVAS IMPLEMENTAÇÕES

O que mais podemos **implementar** na nossa **calculadora**?





NOVAS IMPLEMENTAÇÕES

Passo 1: Implementar as funções trigonométricas e raiz quadrada

Passo 2: Colocar as novas opções no menu

Passo 3: Try except para verificar se o usuário digitou uma opção válida

Passo 4: Condição para receber o número em cada caso

Passo 5: Chamar as novas funções e exibir o resultado para o usuário





PASSO 1: IMPLEMENTANDO AS NOVAS FUNÇÕES

No arquivo que criamos `funcoes_calculadora.py`

```
import math
```



Esta biblioteca vai ser importante para calcular nossas novas funções



PASSO 1: IMPLEMENTANDO AS NOVAS FUNÇÕES

No arquivo que criamos `funcoes_calculadora.py`

```
def raiz_quadrada(numero):  
    resultado = math.sqrt(numero)  
    return resultado
```

```
def seno(numero):  
    resultado = math.sin(numero)  
    return resultado
```

```
def cosseno(numero):  
    resultado = math.cos(numero)  
    return resultado
```

```
def tangente(numero):  
    resultado = math.tan(numero)  
    return resultado
```




PASSO 2: ACRESCENTANDO AO MENU

No arquivo que criamos `main.py`

```
#Estamos dentro do while!  
print("1 - Soma")  
print("2 - Subtração")  
print("3 - Multiplicação")  
print("4 - Divisão")  
print("5 - Exponenciação")  
print("6 - Raiz quadrada")  
print("7 - Seno")  
print("8 - Cosseno")  
print("9 - Tangente")  
print("10 - Sair")
```



PASSO 3: IMPLEMENTANDO O TRY EXCEPT

No arquivo que criamos main.py

```
try:
    opcao = int(input("Escolha uma opção: "))
except ValueError:
    print("Por favor, digite um número.")
    continue

if opcao < 1 or opcao > 10:
    print("Opção inválida. Por favor, escolha uma opção entre 1 e 10.")
    continue

if opcao == 10:
    break
```



PASSO 4: CONDIÇÕES PARA RECEBER OS NÚMEROS

No arquivo que criamos `main.py`

```
if opcao >= 1 and opcao <= 5:  
    numero1 = float(input("Digite o primeiro número: "))  
    numero2 = float(input("Digite o segundo número: "))  
else:  
    numero1 = float(input("Digite o número: "))
```



PASSO 5: CHAMANDO AS FUNÇÕES E EXIBINDO

No arquivo que criamos main.py

```
if opcao == 1:
    resultado = funcoes.soma(numero1, numero2)
    print(f"{numero1} + {numero2} = {resultado}")
elif opcao == 2:
    resultado = funcoes.subtracao(numero1, numero2)
    print(f"{numero1} - {numero2} = {resultado}")
elif opcao == 3:
    resultado = funcoes.multiplicacao(numero1, numero2)
    print(f"{numero1} * {numero2} = {resultado}")
```



PASSO 5: CHAMANDO AS FUNÇÕES E EXIBINDO

No arquivo que criamos main.py

```
elif opcao == 4:
    if numero2 != 0:
        resultado = funcoes.divisao(numero1, numero2)
        print(f"{numero1} / {numero2} = {resultado:.2f}")
    else:
        print("Erro: Divisão por zero não é permitida.")
elif opcao == 5:
    resultado = funcoes.exponenciacao(numero1, numero2)
    print(f"{numero1} ^ {numero2} = {resultado}")
elif opcao == 6:
    if numero1 < 0:
        numero1 = numero1 * -1
    resultado = funcoes.raiz_quadrada(numero1)
    print(f"√{numero1} = {resultado:.2f}")
```




PASSO 5: CHAMANDO AS FUNÇÕES E EXIBINDO

No arquivo que criamos main.py

```
elif opcao == 7:
    resultado = funcoes.seno(numero1)
    print(f"sin({numero1}) = {resultado:.2f}")
elif opcao == 8:
    resultado = funcoes.cosseno(numero1)
    print(f"cos({numero1}) = {resultado:.2f}")
elif opcao == 9:
    resultado = funcoes.tangente(numero1)
    print(f"tan({numero1}) = {resultado:.2f}")
```



NÓS CONSEGUIMOS FAZER UMA CALCULADORA DO ZERO!





Agora você é oficialmente uma
CODIFICADORA,
e pode codificar um mundo de possibilidades! =)



RECOMENDAÇÕES

- site **W3Schools**, onde é possível acessar a documentação Python;
- canal do Youtube do **prof. Guanabara**;
- cursos **Udemy**.



Continue estudando!



@CODIFICADORAS.UTFPR
Mulheres que **codificam** o mundo!





FORMULÁRIO DE FEEDBACK

